GitHub Actions → Azure OIDC Setup Guide

Nov 1, 2026

What you're building

You are letting a GitHub Actions workflow log in to Azure and run Terraform without storing any password or secret in GitHub. This is called OpenID Connect (OIDC) or "workload identity federation". Plain-English version: GitHub signs a note that says "this job is running from repo X, environment Y", Azure checks the note against a list you set up, and if it matches Azure hands back a short-lived access token. The token expires in about an hour, so there is nothing to leak or rotate.

```
GitHub Actions job                          GitHub token service
(Terraform runs here)   --(1) request -->   token.actions.githubusercontent.com
       |                <--(2) signed OIDC token (contains the "subject")
       |
       |--(3) present OIDC token -->  Microsoft Entra ID
       |                              checks it against the App registration's
       |                              federated credentials: issuer + subject +
       |                              audience must match EXACTLY (case-sensitive)
       |<--(4) short-lived Azure access token
       |
       '--(5) use Azure token -->  Azure (Resource Manager + Terraform state storage)
```

Read top to bottom: the job gets a signed token from GitHub (1, 2), hands it to Entra, which checks it against a federated credential and returns an Azure token (3, 4), and Terraform uses that token to reach Azure (5).

There are eight setup steps, and the order matters because each one needs something from the one before it:

1. Get your GitHub numeric IDs (Azure's form needs numbers, not names).
2. Create an Azure App registration (the identity Terraform will run as).
3. Add federated credentials to it (the list Azure checks GitHub's note against).
4. Create the storage account and container that will hold Terraform state.
5. Give the identity roles (permission to create resources and to write state).
6. Register resource providers on the subscription (opt in to Key Vault, Storage, etc.).
7. Configure GitHub: environments, variables, and the workflow file.
8. Push to the `dev` branch and watch it run.

Everything in this guide uses the real values from one working setup (account `Starlyvil`, repo `hybrid-ingestion-platform`). Substitute your own wherever you see them.

Before you start

You need an Azure subscription where you can create resources and assign roles, a GitHub repo that holds your Terraform code, and either the Azure portal in a browser or the `az` CLI (every step below gives both).

- ☐ You can sign in to the Azure portal and open your subscription.
- ☐ You are Owner (or Contributor plus User Access Administrator) on that subscription. You need this to assign roles and register providers.
- ☐ You have the GitHub repo and can open its Settings tab (you are admin on it).
- ☐ Optional: `az` CLI installed and signed in (`az login`), and `gh` CLI installed and signed in (`gh auth login`).

Keep a scratch note open. You will collect these values as you go and paste them into GitHub in Step 7. The right column shows the real values from the worked example; yours will differ.

Value	Where it comes from	Example value
GitHub account name (exact case)	Your GitHub profile	`Starlyvil`
GitHub repo name	Your repo	`hybrid-ingestion-platform`
Owner / Organization ID (number)	Step 1	`31422222`
Repository ID (number)	Step 1	`1437688551`
Tenant ID	Step 2 (App registration overview)	`2dbe89f7-4a82-4950-8077-64ea9809c62e`
Client (application) ID	Step 2 (App registration overview)	`4b0e464d-9fa9-4540-9095-853f79bdf0b7`
Subscription ID	Azure portal, Subscriptions	`147103a9-1f23-4c65-ae33-96dac6063586`
State resource group	Step 4	`rg-ingest-tfstate`
State storage account	Step 4	`dfstorageclass`
State container	Step 4	`tfstate`

None of these values are secrets. IDs only identify things; they do not grant access on their own.

Step 1: Get your GitHub numeric IDs

Azure's federated-credential form now requires two numbers: your account's ID and your repo's ID. Why: names can be renamed, numbers cannot, so Azure builds the match string from the numbers. If you type the name instead, the form says "Must be a numeric value".

You need two numbers. In the worked example they are Owner ID `31422222` and Repository ID `1437688551`. Pick any one of the methods below.

Method A: gh CLI (fastest, works for private repos)

1. Open a terminal.
2. Type the command below, replacing the account and repo names with yours, and press Enter.

```
gh api repos/Starlyvil/hybrid-ingestion-platform --jq '{owner_id: .owner.id, repo_id: .id}'
```

3. You get something like `{"owner_id":31422222,"repo_id":1437688551}`. Write both numbers down.

Method B: view the page source in your browser (no tools needed)

1. Sign in to GitHub and open your repo's main page.
2. Press `Ctrl+U` (Windows/Linux) or `Cmd+Option+U` (Mac) to view the page source.
3. Press `Ctrl+F` and search for `octolytics-dimension`.
4. Find `octolytics-dimension-user_id` (or `-owner_id`). The number in `content="..."` is your Owner / Organization ID.
5. Find `octolytics-dimension-repository_id`. That number is your Repository ID.

Method C: the GitHub API in a browser (public info only)

1. Open `https://api.github.com/users/Starlyvil` (replace with your account name). The `"id"` field is your Owner ID.
2. Open `https://api.github.com/repos/Starlyvil/hybrid-ingestion-platform`. The top-level `"id"` field is the Repository ID.
3. If the repo is private, step 2 shows `404 Not Found`. That is expected: your browser does not send your GitHub login to the API. Use Method A or B, or add a token as below.

Method C with a token (for private repos)

1. Get a token. Easiest: `gh auth token` prints the token the gh CLI already has. Otherwise create one: GitHub, top-right profile picture, Settings, Developer settings, Personal access tokens, Fine-grained tokens, Generate new token. Set Repository access to "Only select repositories" and pick the repo (or "All repositories"), and under Repository permissions set Metadata to Read-only. A classic token also works if it has the `repo` scope.
2. Run, replacing `<token>`:

```
curl -H "Authorization: Bearer <token>" https://api.github.com/repos/Starlyvil/hybrid-ingesti
on-platform
```

3. Read `"id"` (repo) and `"owner": {"id": ...}` (owner).

If the token works on `https://api.github.com/user` but gives 404 on the repo, the token cannot see private repos. Causes: a classic token without `repo` scope, a fine-grained token set to "Public repositories (read-only)", or the wrong Resource owner. 401 means the token itself is invalid; 404 means a valid token that cannot see that repo.

Treat tokens like passwords: never paste them into chat, docs or code, and revoke them (Settings, Developer settings, Delete) when you are done.

Step 2: Create the App registration

An App registration is the identity that Terraform will run as in Azure. Think of it as a user account for a robot. Unlike a normal user it has no password here: in Step 3 you will tell it to trust GitHub instead. Creating it gives you two values you need later: the Client ID and the Tenant ID.

Portal

1. Open portal.azure.com and sign in.
2. In the top search bar type `Microsoft Entra ID` and click it.
3. In the left menu click **App registrations**.
4. Click **+ New registration** at the top.
5. In **Name** type a name you will recognise, for example `github-hybrid-ingestion-terraform`.
6. Leave **Supported account types** on "Accounts in this organizational directory only".
7. Leave **Redirect URI** empty. It is for web sign-in and does not apply here.
8. Click **Register**.
9. You land on the app's **Overview** page. Copy **Application (client) ID** and **Directory (tenant) ID** into your scratch note. In the worked example these are `4b0e464d-9fa9-4540-9095-853f79bdf0b7` and `2dbe89f7-4a82-4950-8077-64ea9809c62e`.

Do not go to Certificates & secrets to create a client secret. Not having a secret is the whole point of this setup.

CLI

1. Create the app and note the `appId` it prints:

```
az ad app create --display-name github-hybrid-ingestion-terraform --query appId -o tsv
```

2. Create the service principal for it. The app registration is the definition; the service principal is the account in your tenant that roles can be assigned to. The portal does this for you automatically; the CLI does not.

```
az ad sp create --id <appId>
```

3. Get your tenant ID and subscription ID:

```
az account show --query "{tenantId: tenantId, subscriptionId: id}" -o table
```

Step 3: Add federated credentials

A federated credential is one line in a trust list: "if GitHub presents a note whose subject is exactly this string, accept it as this app". The subject string encodes the repo and either the pull-request event or a GitHub environment name. You need three of them, one per job type in the workflow.

Name	Entity type	Value	Subject Azure will generate
`GITHUB_CICD_PR`	Pull request	(none)	`repo:Starlyvil@31422222/hybrid-ingestion-platform@1437688551:pull_request`
`env-dev`	Environment	`dev`	`repo:Starlyvil@31422222/hybrid-ingestion-platform@1437688551:environment:dev`
`env-prod`	Environment	`prod`	`repo:Starlyvil@31422222/hybrid-ingestion-platform@1437688551:environment:prod`

Why these three: any job that declares `environment: dev` in the workflow gets the subject `...:environment:dev`, no matter which branch or event triggered it. The PR plan job declares no environment, so its subject is `...:pull_request`. (A push job with no environment would instead get `...:ref:refs/heads/<branch>`, entity type Branch. This workflow does not use that.)

Portal (repeat once per row of the table)

1. Open your App registration (Microsoft Entra ID, App registrations, click its name).
2. In the left menu click **Certificates & secrets**.
3. Click the **Federated credentials** tab.
4. Click **+ Add credential**.
5. In **Federated credential scenario** choose **GitHub Actions deploying Azure resources**.
6. In **Organization** type your GitHub account name exactly as GitHub shows it, including capital letters: `Starlyvil`. For a personal account this field still says "Organization"; put your username there.
7. In **Organization ID** type the Owner ID number from Step 1: `31422222`.
8. In **Repository** type the repo name: `hybrid-ingestion-platform`.
9. In **Repository ID** type the Repository ID number from Step 1: `1437688551`.
10. In **Entity type** choose **Pull request** for the first row, or **Environment** for the other two.
11. For Environment rows, a **Based on selection** (GitHub environment name) box appears. Type `dev` or `prod`.
12. Check the **Subject identifier** preview. It must match the table above character for character.
13. In **Name** type the row's name, for example `env-dev`.
14. Leave **Issuer** as `https://token.actions.githubusercontent.com` and **Audience** as `api://AzureADTokenExchange`.
15. Click **Add**.

CLI (repeat once per row, changing name and subject)

```
az ad app federated-credential create --id <appId> --parameters '{"name":"env-dev","issuer":"https://token.actions.githubusercontent.com","subject":"repo:Starlyvil@31422222/hybrid-ingestion-platform@1437688551:environment:dev","audiences":["api://AzureADTokenExchange"]}'
```

Check what you have:

```
az ad app federated-credential list --id <appId> --query "[].subject" -o tsv
```

Things that went wrong in the worked example, so you can avoid them:

- A wrong Organization ID (`31832195` instead of `31422222`) gave `AADSTS700213: No matching federated identity record found`. Re-check both numbers against Step 1.
- Typing `starlyvil` in lowercase gave `AADSTS7002138: ... matches with case-insensitive comparison, but not with case-sensitive`. The subject is case-sensitive; copy the name exactly as GitHub displays it.
- Having only the Pull request credential while the deploy job used `environment: dev` also gave AADSTS700213. Each distinct subject needs its own credential.
- Duplicates with different names but the same subject are harmless, but Entra refuses to add a second credential with the same issuer and subject. Delete extras. The limit is 20 per app.
- A new or changed credential can take about a minute to take effect. Wait before re-running.

Step 4: Create the Terraform state storage and container

Terraform remembers what it has built in a "state" file. In a pipeline that file must live somewhere shared, so it lives in an Azure storage container. Terraform cannot create the place it stores its own state, so you create the resource group, storage account and container by hand once. Terraform creates the state files inside it (`dev.tfstate` , `prod.tfstate`) by itself.

In the worked example: resource group `rg-ingest-tfstate` , storage account `dfstorageclass` , container `tfstate` . Storage account names must be 3 to 24 lowercase letters and digits and unique across all of Azure, so pick your own.

Portal

1. In the top search bar type `Storage accounts` and click it.
2. Click **+ Create**.
3. **Subscription**: pick yours. **Resource group**: click **Create new**, type `rg-ingest-tfstate` , click OK.
4. **Storage account name**: type your unique name, for example `dfstorageclass` .
5. **Region**: pick the region you deploy to. **Performance**: Standard. **Redundancy**: Locally-redundant storage (LRS) is enough.
6. Click **Review + create**, then **Create**. Wait for "Your deployment is complete" and click **Go to resource**.
7. In the storage account's left menu expand **Data storage** and click **Containers**.
8. Click **+ Container**.
9. **Name**: type `tfstate` . Leave access level on **Private**. Click **Create**.

CLI

```
az group create -n rg-ingest-tfstate -l <region>
az storage account create -n dfstorageclass -g rg-ingest-tfstate -l <region> --sku Standard_LRS
az storage container create -n tfstate --account-name dfstorageclass --auth-mode login
```

`--auth-mode login` uses your Entra ID sign-in instead of a storage key. If it says you are not authorised, give yourself Storage Blob Data Contributor on the account (same steps as Step 5) and retry.

How the repo refers to it

The backend block in `infra/versions.tf` names the resource group and container and tells Terraform to sign in with Entra ID rather than a storage key:

```
backend "azurerm" {
  resource_group_name = "rg-ingest-tfstate"
  container_name      = "tfstate"
  use_azuread_auth    = true
}
```

The storage account name and the state file name are passed in at `terraform init` time by the workflow, so one backend block serves both environments:

```
terraform init \
  -backend-config="storage_account_name=${{ vars.TFSTATE_STORAGE_ACCOUNT }}" \
  -backend-config="key=dev.tfstate"
```

If the container is missing, `terraform init` fails with `404 The specified container does not exist. ContainerNotFound`. Create it and re-run.

Step 5: Assign roles to the identity

The app registration now exists and GitHub can sign in as it, but it is allowed to do nothing. Roles are the permissions. They are given to the identity Terraform runs as, not to the things Terraform will create, so you assign them at a scope that already exists: the whole subscription, or a resource group you created in advance. Without a role the first apply fails with `AuthorizationFailed`; this is the chicken-and-egg step that a human has to do once.

You need two roles, and a third only in one case:

Role	Scope	Why
Contributor	The subscription, or the resource group Terraform deploys into	Lets Terraform create and change resources.
Storage Blob Data Contributor	The state storage account (`dfstoragerageclass`) or its resource group `rg-ingest-tfstate`	`use_azuread_auth = true` reads and writes the state file with Entra ID. Contributor does not include blob data access, so this is a separate role.
Role Based Access Control Administrator (or User Access Administrator)	Only if your Terraform code itself creates role assignments	Contributor cannot grant roles to others.

Assigning at a resource group is tighter but means Terraform can only build inside that group. The worked example's provider comment notes the pipeline identity is Contributor on the resource group only, which is also why it cannot purge a deleted Key Vault (`purge_soft_delete_on_destroy = false`). Assignments at a resource group inherit to every resource created inside it later.

Portal (repeat once per row)

1. Open the scope. For the subscription: search `Subscriptions`, click yours. For a resource group: search `Resource groups`, click it. For the storage account: search `Storage accounts`, click `dfstorageclass`.
2. In the left menu click **Access control (IAM)**.
3. Click **+ Add**, then **Add role assignment**.
4. On the **Role** tab, in the search box type the role name (for example `Contributor`), click the matching row, then click **Next**. (Contributor is under the **Privileged administrator roles** tab.)
5. On the **Members** tab, next to **Assign access to**, choose **User, group, or service principal**.
6. Click **+ Select members**. In the search box type your app's name (for example `github-hybrid-ingestion-terraform`), click it in the list, then click **Select**.
7. Click **Review + assign**, then **Review + assign** again.
8. Back on the IAM page, click **Role assignments** and confirm the app appears with the role.

CLI

```
# Contributor on the whole subscription
az role assignment create --assignee <appId> --role Contributor \
  --scope /subscriptions/<subscriptionId>

# or Contributor on one resource group only
az role assignment create --assignee <appId> --role Contributor \
  --scope /subscriptions/<subscriptionId>/resourceGroups/<rg-name>

# Blob data access to the state storage account
az role assignment create --assignee <appId> --role "Storage Blob Data Contributor" \
  --scope /subscriptions/<subscriptionId>/resourceGroups/rg-ingest-tfstate/providers/Microsoft.Storage/storageAccounts/dfstorageclass
```

Check what the identity has:

```
az role assignment list --assignee <appId> --all -o table
```

Role assignments can take a few minutes to propagate. If the first run fails with an authorisation error right after assigning, wait and re-run.

Step 6: Register resource providers

The worked example's first apply got this far and then failed while creating the Key Vault:

```
409 Conflict MissingSubscriptionRegistration: The subscription is not registered to use namespace 'Microsoft.KeyVault'
```

What a resource provider is. Every Azure service is delivered by a "resource provider", named by a namespace: `Microsoft.KeyVault` runs Key Vault, `Microsoft.Storage` runs storage accounts, `Microsoft.OperationalInsights` runs Log Analytics, and so on. A subscription has to opt in to each provider

before it can create resources of that type. This is called registering the provider. It is free, it is one-time per subscription, and it lets the provider set up its own plumbing for your subscription. A new subscription has only a handful of providers registered by default.

Why you have never seen this before. When you create something by clicking in the portal, the portal registers the provider for you in the background. Pipelines and Terraform call the Azure API directly, so nobody does that for them.

Why Terraform did not do it for you. The azurerm provider does try to register providers automatically. But registration is an action on the subscription itself (`Microsoft.KeyVault/register/action` at subscription scope). An identity that only has Contributor on a resource group has no rights at subscription scope, so the attempt fails and the real error shows up later when the resource is created. A human with subscription-level rights registers it once; after that every pipeline run is fine.

Portal

1. In the top search bar type `Subscriptions` and click it.
2. Click your subscription.
3. In the left menu expand **Settings** and click **Resource providers**.
4. In the filter box type `Microsoft.KeyVault`.
5. Click the row so it is selected. Its status column says **NotRegistered**.
6. Click **Register** at the top.
7. Wait and refresh. The status goes to **Registering** and then **Registered** after about 1 to 2 minutes.
8. Repeat for each provider in the list below.

CLI

```
az account set -s <subscriptionId>
az provider register -n Microsoft.KeyVault --wait
az provider show -n Microsoft.KeyVault --query registrationState -o tsv
```

The last command should print `Registered`.

Providers this project needs. Register all of these now so you do not hit the error one provider at a time:

Provider	Used for
`Microsoft.Storage`	Storage accounts and the data-lake containers
`Microsoft.KeyVault`	Key Vault
`Microsoft.OperationalInsights`	Log Analytics workspace
`Microsoft.Insights`	Diagnostic settings that send logs to Log Analytics
`Microsoft.EventGrid`	Possibly later, for blob-created events
`Microsoft.DataFactory`	Possibly later, for Data Factory pipelines

```
for p in Microsoft.Storage Microsoft.KeyVault Microsoft.OperationalInsights Microsoft.Insight
s; do
  az provider register -n $p --wait
done
az provider list --query "[?registrationState=='Registered'].namespace" -o tsv
```

After registering, re-run the workflow. Terraform skips the resources it already created and continues from where it failed.

Step 7: Configure GitHub

GitHub needs three things: environments named `dev` and `prod` (the federated credentials from Step 3 match on these names), the four ID values as variables, and a workflow that asks for an OIDC token and hands it to Terraform.

7a. Create the environments

1. Open your repo on GitHub and click **Settings** (the tab on the right; if you do not see it you are not admin).
2. In the left menu click **Environments**.
3. Click **New environment**, type `dev`, click **Configure environment**. No protection rules are needed for dev.
4. Go back to Environments, click **New environment**, type `prod`, click **Configure environment**.
5. For `prod`, tick **Required reviewers**, add yourself, click **Save protection rules**. Now a prod deploy pauses until someone approves it.

The environment name must match the federated credential's GitHub environment name exactly (`dev`, `prod`).

7b. Add the variables

These are plain variables, not secrets, because IDs are not sensitive. Each environment needs its own copy.

Variable name	Value (worked example)
`AZURE_CLIENT_ID`	`4b0e464d-9fa9-4540-9095-853f79bdf0b7`
`AZURE_TENANT_ID`	`2dbe89f7-4a82-4950-8077-64ea9809c62e`
`AZURE_SUBSCRIPTION_ID`	`147103a9-1f23-4c65-ae33-96dac6063586`
`TFSTATE_STORAGE_ACCOUNT`	`dfstorageclass`

1. Settings, Environments, click `dev`.
2. Scroll to **Environment variables** and click **Add environment variable**.
3. Type the **Name** and **Value** from the table, click **Add variable**. Repeat for all four.
4. Repeat steps 1 to 3 for `prod` (use prod's own client ID if you created a separate app for prod).

5. Also add the same four as **repository** variables: Settings, **Secrets and variables**, **Actions**, click the **Variables** tab, **New repository variable**. Why: the PR `plan` job reads `vars.*` but declares no `environment:`, so it only sees repository-level variables. Without this step the plan job gets empty values and fails to sign in.

With `gh`:

```
gh variable set AZURE_CLIENT_ID --env dev --body "4b0e464d-9fa9-4540-9095-853f79bdf0b7"
gh variable set AZURE_CLIENT_ID --body "4b0e464d-9fa9-4540-9095-853f79bdf0b7"   # repo level
```

7c. The workflow file

The repo uses `.github/workflows/infra.yml` with a `plan` job on pull requests and `deploy-dev` / `deploy-prod` jobs on pushes that call a reusable `deploy.yml` with `environment: dev` or `prod`. The parts that make OIDC work:

```
permissions:
  id-token: write   # lets the job ask GitHub for an OIDC token
  contents: read

env:
  ARM_USE_OIDC: true
  ARM_CLIENT_ID: ${{ vars.AZURE_CLIENT_ID }}
  ARM_TENANT_ID: ${{ vars.AZURE_TENANT_ID }}
  ARM_SUBSCRIPTION_ID: ${{ vars.AZURE_SUBSCRIPTION_ID }}
```

`id-token: write` is the permission that lets the job request the signed note from GitHub. `ARM_USE_OIDC: true` tells the Terraform azurerm provider and backend to use that note instead of a client secret.

Who reads the `ARM_*` variables (it is not `setup-terraform`)

Two different HashiCorp pieces appear in the workflow, and they do different jobs:

Piece	What it is	Where it comes from	Reads `ARM_*`?
`hashicorp/setup-terraform@v3`	A GitHub Action that downloads the `terraform` program onto the runner	The `uses:` line in the workflow	No
`hashicorp/azurerm`	A Terraform plugin (provider) that knows how to talk to Azure	`source = "hashicorp/azurerm"` in `infra/versions.tf`; downloaded by `terraform init`	Yes

What happens in the job, in order:

1. `uses: hashicorp/setup-terraform@v3` installs Terraform. That is all it does.
2. `terraform init` reads `infra/versions.tf`, downloads the azurerm provider, and the **azurerm backend** reads the `ARM_*` variables to sign in and reach the `tfstate` container.

3. `terraform plan` / `apply`: the **azurerm provider** reads the same `ARM_*` variables to sign in and create resources.

The job-level `env:` block sets these variables for every step, but only the Terraform commands use them. Swapping `setup-terraform` for another way of installing Terraform changes nothing about this setup.

Which names are fixed and which are your choice:

Part	Fixed?
`ARM_USE_OIDC`, `ARM_CLIENT_ID`, `ARM_TENANT_ID`, `ARM_SUBSCRIPTION_ID` (left side)	Fixed. The azurerm provider looks for exactly these names. Rename one (for example `AZ_CLIENT_ID`) and it is ignored.
`vars.AZURE_CLIENT_ID` etc. (right side)	Your choice. These are your own GitHub variable names; `AZURE_*` is just a common convention.

`ARM_USE_OIDC: true` makes the provider request the OIDC token using two values GitHub sets automatically on the runner (`ACTIONS_ID_TOKEN_REQUEST_URL` and `ACTIONS_ID_TOKEN_REQUEST_TOKEN`). GitHub only sets them when the workflow has `permissions: id-token: write`. The same settings can also go in code (`use_oidc`, `client_id`, `tenant_id`, `subscription_id` on the provider and backend blocks), but environment variables keep IDs out of the code.

The deploy jobs are gated on branch:

```
deploy-dev:
  if: github.event_name == 'push' && github.ref == 'refs/heads/dev'
  uses: ./.github/workflows/deploy.yml
  with:
    environment: dev

deploy-prod:
  if: github.event_name == 'push' && github.ref == 'refs/heads/prod'
  uses: ./.github/workflows/deploy.yml
  with:
    environment: prod
```

Do not put `needs: deploy-dev` on `deploy-prod`. On a push to `prod`, deploy-dev is skipped (wrong branch), and a job whose `needs` was skipped is also skipped, so prod would never deploy.

Step 8: Run and verify

Push a commit to the `dev` branch, or re-run the last workflow from the Actions tab, and watch the job sign in, configure the backend and create resources.

1. Push to `dev`:

```
git checkout dev
git commit --allow-empty -m "Trigger infra deploy"
git push origin dev
```

2. On GitHub click the **Actions** tab, click the newest run, then click the `deploy-dev` job.
3. Expand the **terraform init** step. Success looks like `Successfully configured the backend "azurerm"!` followed by `Terraform has been successfully initialized!`.
4. Expand the **terraform apply** step. You see one `Creating...` then `Creation complete` line per resource, and at the end `Apply complete! Resources: N added, 0 changed, 0 destroyed.`

In the worked example's first successful run the apply created the resource group `rg-ingest-dev`, a storage account with the containers `landing`, `raw`, `curated`, `archive` and `quarantine`, a diagnostic setting sending logs to Log Analytics, and a Key Vault.

Verify in Azure

1. Portal: search `Resource groups`, click `rg-ingest-dev`, and confirm the resources listed above are there.
2. Portal: search `Storage accounts`, click `dfstorageclass`, **Data storage**, **Containers**, click `tfstate`. A file `dev.tfstate` now exists. Terraform created it on its own.

```
az resource list -g rg-ingest-dev -o table
az storage blob list --account-name dfstorageclass -c tfstate --auth-mode login --query "[].name" -o tsv
```

To test prod, merge or push to the `prod` branch. The run pauses at the `prod` environment until a required reviewer clicks **Review deployments**, ticks `prod` and clicks **Approve and deploy**. A `prod.tfstate` then appears beside `dev.tfstate`.

To test the PR path, open a pull request against `dev`. The `plan` job signs in with the `:pull_request` credential and runs a plan without applying anything.

Troubleshooting

Every error below was hit in the worked example, in this order, except the last few, which are the most likely next ones.

Error you see	Where	What it means	Fix
`Must be a numeric value`	Azure portal, federated credential form	You typed a name in Organization ID or Repository ID.	Use the numbers from Step 1.
`404 Not Found` from `api.github.com/repos/...`	Browser or curl	The repo is private and the request carried no login, or the token cannot see private repos.	Use `gh api` (Step 1 Method A), view source (Method B), or a token with private-repo access.
`401 Bad credentials` from api.github.com	curl	The token is invalid, expired or mistyped.	Generate a new token.
`AADSTS700213: No matching federated identity record found`	GitHub Actions log, terraform init / Azure login	GitHub's subject does not match any credential on the app: wrong numeric ID, or no credential for that entity type (for example a job with `environment: dev` but only a Pull request credential).	Compare the subject in the error with the Step 3 table; add or fix the credential; wait a minute.
`AADSTS7002138: ... matches with case-insensitive comparison, but not with case-sensitive`	GitHub Actions log	The Organization or Repository name has different capitalisation from GitHub.	Retype it exactly as GitHub shows it (`Starlyvil`).
`404 ContainerNotFound: The specified container does not exist`	`terraform init`	The `tfstate` container is missing from the storage account.	Create it (Step 4).
`409 MissingSubscriptionRegistration: ... namespace 'Microsoft.KeyVault'`	`terraform apply`	The subscription has not opted in to that resource provider.	Register it as a human with subscription rights (Step 6), then re-run.
`403 AuthorizationPermissionMismatch` on the state blob	`terraform init` or `plan`	The identity can see the storage account but not its blob data.	Assign Storage Blob Data Contributor on the storage account (Step 5).
`AuthorizationFailed: The client ... does not have authorization to perform action ...`	`terraform apply`	The identity has no Contributor role at the scope it is writing to.	Assign Contributor on the subscription or the target resource group (Step 5).
Variables are empty in the `plan` job	GitHub Actions log	The job has no `environment:` so it cannot see environment variables.	Add the four variables at repository level (Step 7b).
`deploy-prod` always skipped	Actions run summary	It has `needs: deploy-dev`, which was skipped on the prod branch.	Remove `needs: deploy-dev` (Step 7c).

To see the exact subject GitHub sent, open the failing job log and expand the terraform init step. The AADSTS message quotes it, so you can compare it with the Step 3 table character by character.

Quick reference

All the values and commands from this guide in one place, with the worked example's values. Replace them with yours.

Item	Value
GitHub account	`Starlyvil` (case-sensitive)
Repo	`hybrid-ingestion-platform`
Owner ID	`31422222`
Repository ID	`1437688551`
Tenant ID	`2dbe89f7-4a82-4950-8077-64ea9809c62e`
Client ID	`4b0e464d-9fa9-4540-9095-853f79bdf0b7`
Subscription ID	`147103a9-1f23-4c65-ae33-96dac6063586`
Issuer	`https://token.actions.githubusercontent.com`
Audience	`api://AzureADTokenExchange`
Subject, PR job	`repo:Starlyvil@31422222/hybrid-ingestion-platform@1437688551:pull_request`
Subject, dev	`repo:Starlyvil@31422222/hybrid-ingestion-platform@1437688551:environment:dev`
Subject, prod	`repo:Starlyvil@31422222/hybrid-ingestion-platform@1437688551:environment:prod`
State RG / account / container	`rg-ingest-tfstate` / `dfstorageclass` / `tfstate`
GitHub variables (env + repo level)	`AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, `TFSTATE_STORAGE_ACCOUNT`

```
# 1. GitHub IDs
gh api repos/Starlyvil/hybrid-ingestion-platform --jq '{owner_id: .owner.id, repo_id: .id}'

# 2. App registration + service principal
APP_ID=$(az ad app create --display-name github-hybrid-ingestion-terraform --query appId -o t
sv)
az ad sp create --id $APP_ID

# 3. Federated credentials (repeat for pull_request, environment:dev, environment:prod)
az ad app federated-credential create --id $APP_ID --parameters '{"name":"env-dev","issue
r":"https://token.actions.githubusercontent.com","subject":"repo:Starlyvil@31422222/hybrid-in
gestion-platform@1437688551:environment:dev","audiences":["api://AzureADTokenExchange"]}'
az ad app federated-credential list --id $APP_ID --query "[].subject" -o tsv

# 4. State storage
az group create -n rg-ingest-tfstate -l <region>
az storage account create -n dfstorageclass -g rg-ingest-tfstate -l <region> --sku Standard_L
RS
az storage container create -n tfstate --account-name dfstorageclass --auth-mode login

# 5. Roles
SUB=<subscriptionId>
az role assignment create --assignee $APP_ID --role Contributor --scope /subscriptions/$SUB
az role assignment create --assignee $APP_ID --role "Storage Blob Data Contributor" \
  --scope /subscriptions/$SUB/resourceGroups/rg-ingest-tfstate/providers/Microsoft.Storage/st
orageAccounts/dfstorageclass

# 6. Resource providers
az account set -s $SUB
for p in Microsoft.Storage Microsoft.KeyVault Microsoft.OperationalInsights Microsoft.Insight
s; do az provider register -n $p --wait; done

# 7. GitHub variables (repeat per variable; once with --env dev, once with --env prod, once w
ith no --env)
gh variable set AZURE_CLIENT_ID --env dev --body "$APP_ID"

# 8. Run
git push origin dev
```